



Figure 1: Cassandra NoSQL database

## Clustering in Cassandra NoSQL

There are four types of partitioners in Cassandra NoSQL, and it is very important to select the appropriate one for a particular application.

- **Random partitioner:** The random partitioner (RP) is used to distribute the key-value pairs randomly to incorporate effective load balancing. In addition to load balancing, this partitioner implements the dynamic hash functions for the security and integrity of the application. It is the default partitioner in Cassandra for all the applications. RP decides and implements the scheme for organising the keys in the node ring of the Cassandra cluster.
- **Order preserving partitioner (OPP):** This type of partitioner implements the distribution of key-value pairs in the classical method to ensure simplicity and ease of use. The key advantage of this approach is the ability to access fewer nodes rather than multiple nodes in a complex way.
- **Murmur3Partitioner:** This has the same features as the random partitioner (RP) but has a more effective and faster hash than MD5.
- **Byte ordered partitioner:** This type is the order-preserving partitioner, which works on the partition key bytes in a lexicographic way.

The features and key points of Cassandra NoSQL are:

- Scalability
- Fault tolerance
- Consistency
- Column-oriented database
- Distribution Design (DD) is based on Dynamo of Amazon
- Data Model (DM) is based on Bigtable of Google
- Implementation of Dynamo-styled model of replication without any single point of failure
- Powerful data model of Column Family
- Fast linear-scale performance
- Higher throughput
- Flexibility in data storage
- Support for ACID (atomicity, consistency, isolation and durability) transaction properties
- Fast and effective writes to thousands of terabytes without compromising on other parameters

- Support for a number of language drivers including C, C++, Java, Ruby, Go, Python, C# and many others
- Strong developer community for troubleshooting

## Installation of the Cassandra driver with Python

Python is one of the powerful programming languages widely used for Big Data processing, NoSQL databases, cloud infrastructure and related high-performance computing applications.

Python has a Cassandra driver in its repository for linking and interaction with the Cassandra engine.

The Cassandra driver can be installed in real-time from the Internet using the Pip installer of Python with the following command:

```
$ pip install cassandra-driver
```

Alternately, the Cassandra driver for Python can be fetched from the live repository of Python packages <http://pypi.python.org>. The source or compressed file can be downloaded, and then the following command is used after moving it to the folder of the Cassandra driver:

```
$ python setup.py instal
```

## Creating a connection with Cassandra

First of all, an instance of *Cluster* is created before the execution of any query in Cassandra.

This can be done by using the following command:

```
from cassandra.cluster import Cluster
mycluster = Cluster()
```

These instructions create a connection to a Cassandra instance on a local machine (localhost or 127.0.0.1).

A list of IP addresses can be specified for the nodes in your cluster, as follows:

```
from cassandra.cluster import Cluster
mycluster = Cluster(['IP-Address-1', 'IP-Address-2'])
```

To establish the connections and execution of queries, a session is created by using the following instructions:

```
mycluster = Cluster()  
session = cluster.connect()
```

The `connect()` function accepts the optional argument `keyspace` that is used to set the default keyspace for queries in that particular session:

```
mycluster = Cluster()
session = cluster.connect('mydbkeyspace')
```